

Introduction to HTTP APIs

<https://github.com/omkar-ethz/rse-api>

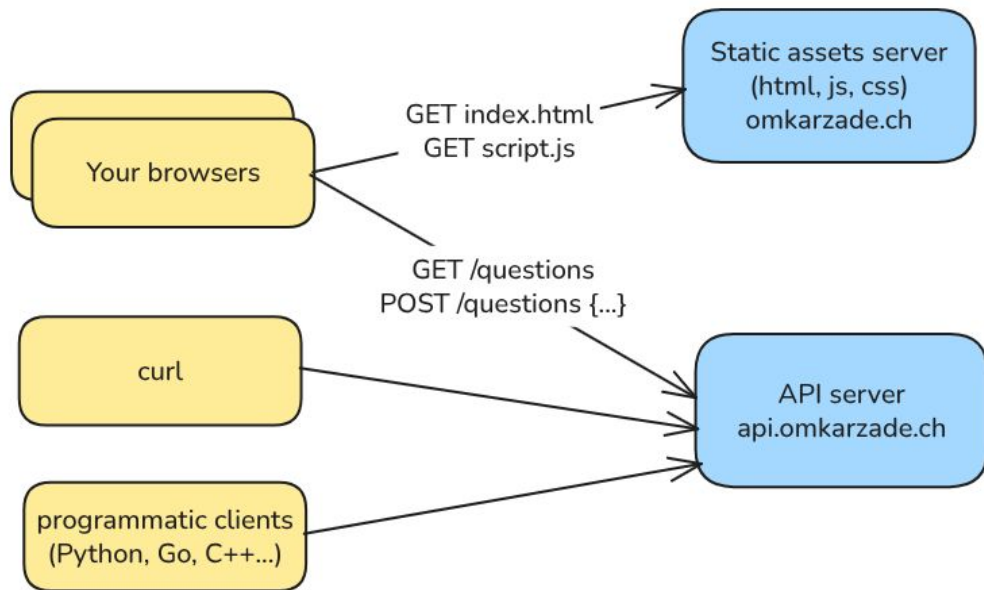
API

- Library
 - `a = np.random.rand(3,2)`
 - Don't need to know implementation details
 - API serves as the contract between you (client code) and the library developers
 - Specifies method arguments, return values, errors
 - Library and client code live and run on the same machine
- Web/HTTP APIs ← this talk
 - Some functionality on a server, shared state
 - Exposed via an API
 - Client and server run on different machines, communicate over a network
 - Multiple clients access / manipulate state via the API

The Questions API

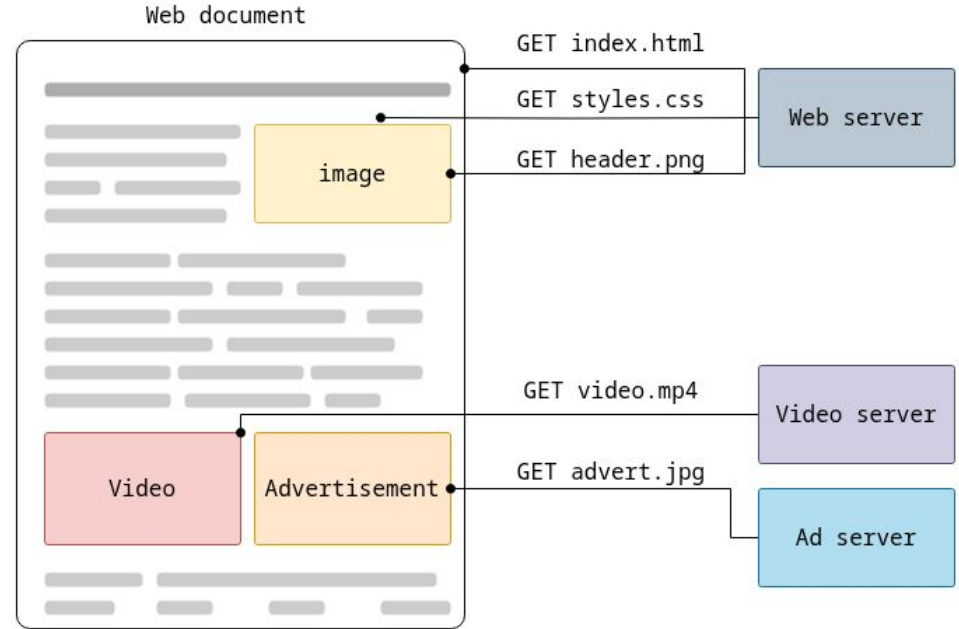
<https://omkarzade.ch/rse-talk>

Note: the website and the API at api.omkarzade.ch/docs will stay active for ~2 months after the talk (until Oct 4 2026), and then shut down.



The HTTP protocol

“HTTP is a protocol for fetching resources such as HTML documents. It is the foundation of any data exchange on the Web and it is a client-server protocol, which means requests are initiated by the recipient, usually the Web browser. A complete document is typically constructed from resources such as text content, layout instructions, images, videos, scripts, and more.” [1]



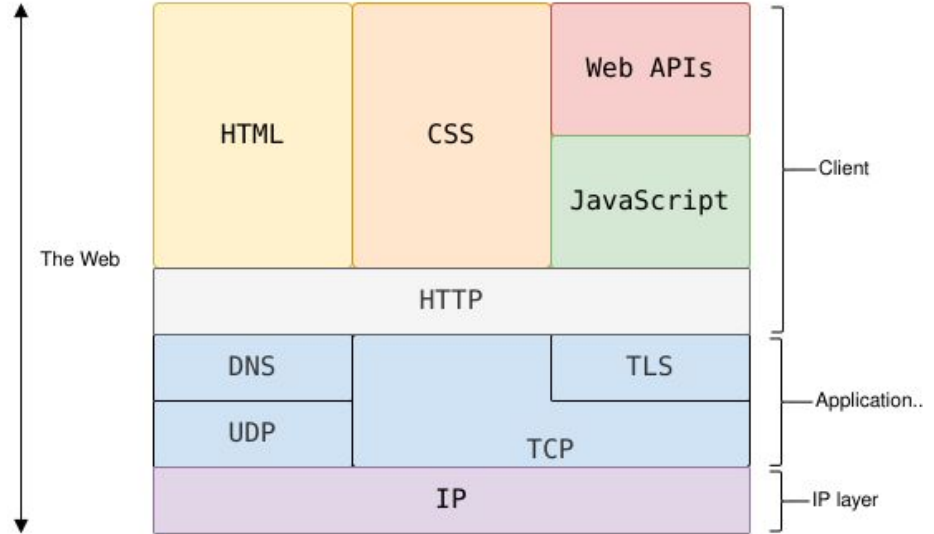
[1] <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>

The HTTP protocol

- HTTP is sent over TCP/IP: a reliable transport protocol
- HTTPS is just HTTP sent over TLS
- Created at CERN ca. 1991

“The WWW project was started to allow high energy physicists to share data, news, and documentation. We are very interested in spreading the web to other areas, and having gateway servers for other data.”

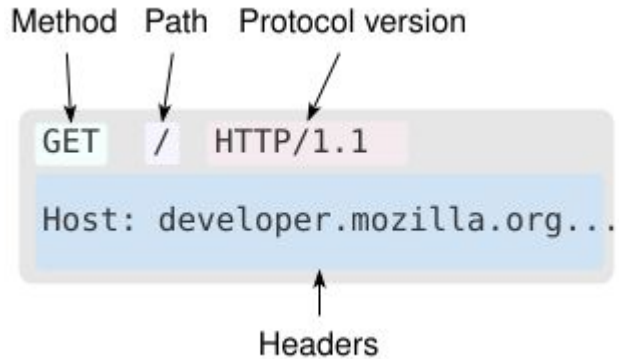
- Tim Berners-Lee, Geneva ^[1]
- HTTP 1.1, HTTP 2, now HTTP 3



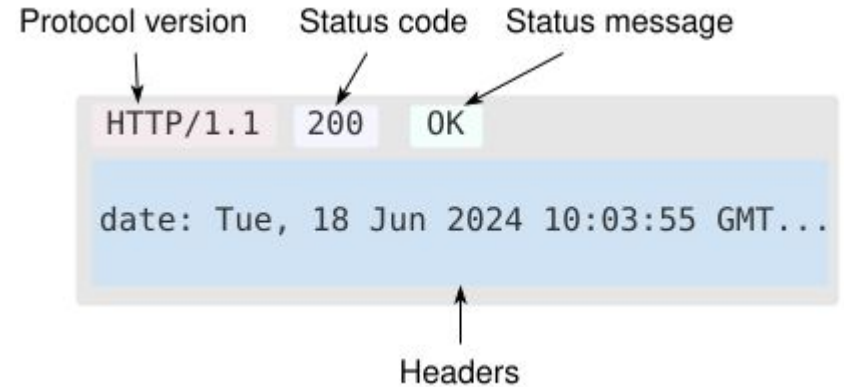
[1] https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Evolution_of_HTTP

HTTP Messages

Request



Response



HTTP Methods

The Questions API provides methods to fetch, post, and edit (upvote) a Question. HTTP defines Methods (aka Verbs) for these operation types

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>

- The GET method requests a representation of the specified resource.
NOTE: Resources to static assets were all GET
- The POST method submits an entity to the specified resource, often causing a change in state or side effects on the server.
- The PATCH method applies partial modifications to a resource.

default

GET	/questions	Get Questions
POST	/questions	Ask Question
PATCH	/questions/{q_id}/upvote	Upvote

HTTP Response Codes

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>

2xx: e.g. 200 OK

3xx: e.g. 301 Redirect

4xx: Client errors

404 Not Found

403 Forbidden

5xx: Server errors

503 Service unavailable

Responses	
Code	Description
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 0, "text": "string", "upvotes": 0 }</pre>
422	Validation Error Media type application/json Example Value Schema <pre>{</pre>

Writing the Questions API

“[FastAPI](#) is a modern, fast (high-performance), web framework for building APIs with Python based on standard Python type hints.”

Walk through the code

Run on localhost

Open the index.html on localhost

If we have time we'll also go into how everything is deployed

Other Python frameworks

- Python is single threaded due to GIL* - you can spawn multiple threads but only one running at a time (even if you have multiple CPUs)
- FastAPI
 - Event loop based concurrency
 - When many of you clicked upvote, FastAPI queued a “task” for each of your requests, and processed them
- Flask
 - Each request handled by a worker (process or thread)
 - With gunicorn: Pre-fork concurrency model

Go

- Compiled
 - Binary includes the go runtime, statically linked
 - Can just copy-paste from your linux laptop to a server running different linux distro
- Goroutines for concurrency
- Standard library quite feature rich: net/http, encoding/json
- Lightweight web frameworks: Gin
 - A service I wrote recently: scicat-s3-broker ^[1]
- Code walkthrough of /questions using net/http

[1] <https://github.com/paulscherrerinstitute/scicat-s3-broker>

TypeScript

- ExpressJS
 - Event loop based concurrency (similar to FastAPI)
 - Many higher-level / opinionated frameworks on top
- SciCat backend - NestJS ^[1]
- SciLog backend - Loopback4 ^[2]
 - considering a rewrite to FastAPI
- SciCat / SciLog frontends - Angular ^[3] ^[2]
- Enables same language on frontend and backend

[1] <https://github.com/SciCatProject/backend>

[2] <https://github.com/paulscherrerinstitute/scilog>

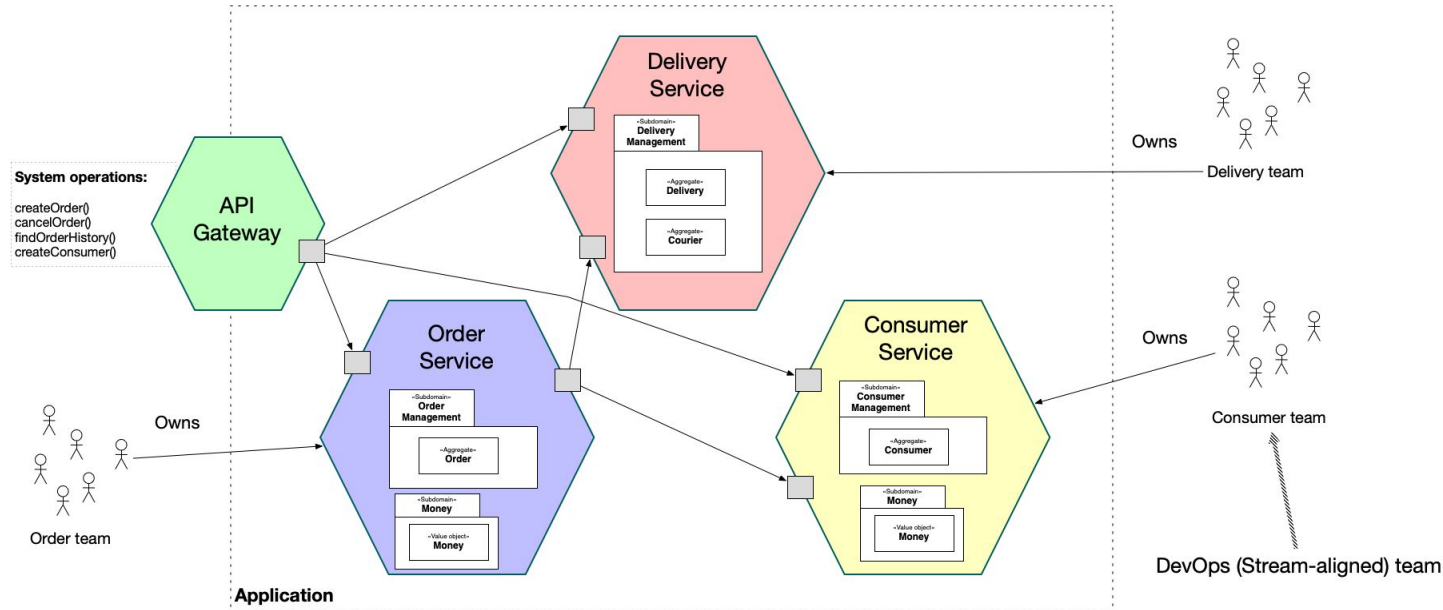
[3] <https://github.com/SciCatProject/frontend>

PSI's SciCat instance: <https://discovery.psi.ch/>

Corresponding API: <https://dacat.psi.ch/explorer>

Microservices

A server may also act as a client. E.g. Service oriented architecture:



Specifying APIs

How do we know which operations are supported on the server? Shape of request / responses? Possible status codes?

The OpenAPI specification

- FastAPI generates it from code i.e. endpoint signature / models
- Other frameworks e.g. Gin: Write spec by hand, generate boilerplate server code from the spec
- You can generate clients (or SDKs) ^[1] in multiple languages from the OpenAPI spec
 - Turns calling HTTP APIs into a remote procedure call. Looks similar to a local library function
 - Instead of
 - `fetch("https://api.omkarzade.ch/questions")` → JSON response
 - `api.GetQuestions()` → `List<Question>` object native to the language

[1] <https://github.com/openapitools/openapi-generator>

Deployment

- API Hosted on a Virtual Machine in Frankfurt
 - DNS record for api.omkarzade.ch → 18.156.76.91
 - Checked out git repo, installed the Python server (with uv), running under systemd on port 8000
 - Caddy (a reverse proxy) terminates TLS on 443, forwards unencrypted HTTP traffic to local port 8000
 - Go: just copy-pasted the binary and execute it
- Static assets served as an S3 website (compiled angular) via CDN

Consider Webhosting from our Linux team!

<https://linux.psi.ch/documentation/services/webhosting/>